Signals and Threads
Quick Overview

sigaction(), kill(), pause(), pthread_kill(), pthread_create(), pthread_exit(), pthread_join(), pthread_sigmask()

Paulo Pedreiras, Pedro Fonseca, DETI/IT/UA

Oct/2026

Linux threads

In Linux/POSIX, a thread is an execution flow inside a process.

- Threads in the same process share:
  - address space;
  - global variables;
  - open file descriptors;
  - signal dispositions.
- Each thread has its own:
  - stack;
  - registers;
  - thread ID;
  - signal mask.

Typical POSIX thread functions

- `pthread_create()` creates a new thread.
- `pthread_exit()` terminates the calling thread.
- `pthread_join()` waits for a thread to terminate.

Creating and waiting for threads

pthread_create()

Creates a new thread that starts executing a given function.

pthread_join()

Waits until a given thread terminates and optionally collects its return value.

```c
#include <pthread.h>
#include <stdio.h>

void *worker(void *arg) {
    printf("Hello from thread\n");
    return NULL;
}

int main(void) {
    pthread_t tid;

    pthread_create(&tid, NULL, worker, NULL);
    pthread_join(tid, NULL);

    return 0;
}
```

Terminating a thread

`pthread_exit()`

Terminates the calling thread and makes a return value available to `pthread_join()`.

- Returning from the thread function is equivalent to calling `pthread_exit()`.
- Calling `exit()` terminates the whole process, not just one thread.
- Calling `pthread_exit()` in `main()` allows other threads to continue running.

```
1 void *worker(void *arg) {
2     pthread_exit((void *) 123);
3 }
```

Linux signals

A signal is an asynchronous notification sent to a process or thread.

- Signals are used to report events such as:
  - user interruption: `SIGINT`;
  - termination request: `SIGTERM`;
  - segmentation fault: `SIGSEGV`;
  - timer expiration: `SIGALRM`.
- A signal may be:
  - ignored;
  - caught by a signal handler;
  - handled by the default action.

Important point

In a multithreaded process, signal disposition is shared by all threads, but each thread has its own signal mask.

Installing a signal handler

sigaction()

Installs or changes the action associated with a signal. It is preferred over the older `signal()` interface.

```c
/* include <signal.h>, <stdio.h> and <unistd.h> */

void handler(int sig) {
    write(STDOUT_FILENO, "Signal received\n", 16);
}

int main(void) {
    struct sigaction sa;

    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask); /*Do not block any other signals while the handler is
      executing */
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL); /* Intercept SIGINT, Interrupt from keyboard*/

    while (1) {
        pause();
    }
}
```

Sending signals to processes

kill()

Sends a signal to a process or to a process group. Despite its name, it does not necessarily kill the process.

```c
#include <signal.h>
#include <unistd.h>

kill(pid, SIGTERM);   // send SIGTERM to process pid
kill(pid, SIGINT);    // send SIGINT to process pid
kill(0, SIGTERM);     // send SIGTERM to own process group
```

- `kill(pid, sig)` sends `sig` to process `pid`.
- `kill(0, sig)` sends `sig` to the caller's process group.
- `kill(-pgid, sig)` sends `sig` to process group `pgid`.

Waiting for a signal

pause()

Suspends the calling process or thread until a signal is delivered.

- pause() returns only after a signal handler has executed.
- It usually returns -1 and sets errno to EINTR.
- It is simple, but care is needed to avoid race conditions.

```c
/* include <signal.h> and <unistd.h> */

volatile sig_atomic_t got_signal = 0;

void handler(int sig) {
    got_signal = 1;
}

int main(void) {
    signal(SIGUSR1, handler);

    while (!got_signal) {
        pause();
    }

    return 0;
}
```

Sending signals to threads

pthread_kill()

Sends a signal to a specific thread in the same process.

- `kill()` targets a process or process group.
- `pthread_kill()` targets a specific POSIX thread.
- The signal action is still process-wide.
- The receiving thread must not have that signal blocked.

```c
#include <pthread.h>
#include <signal.h>

pthread_t tid;

/* Send SIGUSR1 to a specific thread */
pthread_kill(tid, SIGUSR1);
```

Signal masks in threads

pthread_sigmask()

Changes the signal mask of the calling thread.

- Each thread has its own signal mask.
- A blocked signal is not delivered to that thread.
- New threads inherit the signal mask of the creating thread.
- This is useful to control which thread handles signals.

```
1 #include <pthread.h>
2 #include <signal.h>
3
4 sigset_t set;
5
6 sigemptyset(&set);
7 sigaddset(&set, SIGUSR1);
8
9 pthread_sigmask(SIG_BLOCK, &set, NULL);
```

Typical multithreaded signal design

A common strategy is to block signals in all threads and dedicate one thread to signal handling.

1. Main thread blocks selected signals with `pthread_sigmask()`.
2. New worker threads inherit this blocked signal mask.
3. A signal-handling thread waits for signals (`sigwait()` and relatives).
4. Other threads do normal work without asynchronous signal interruption.

Why this is useful

It avoids having signals delivered unpredictably to arbitrary worker threads.

Relevant functions

- `pthread_sigmask()` controls per-thread signal masks.
- `sigaction()` defines what happens when a signal is handled.
- `pthread_kill()` sends a signal to a specific thread.
- `kill()` sends a signal to a process or process group.
- `sigwait()`, `sigwaitinfo()` or `sigtimedwait())` wait for a signal.

Keyboard signal mappings

- The Linux terminal driver intercepts specific keystrokes and translates them into POSIX signals that sent to the foreground process.
- Some common and useful standard keyboard-to-signal mappings are:

Keystroke	Signal	Name	Action
`CTRL + C`	`SIGINT`	Interrupt	Terminates process gracefully. Programs can catch this to clean up before exiting.
`CTRL + Z`	`SIGTSTP`	Terminal Stop	Suspends (pauses) the process and pushes it to the background. The process can be resumed later using fg (foreground) or bg (background) commands.
`CTRL + \`	`SIGQUIT`	Quit	Forces process to terminate, leaving a core dump (for debug).

Summary

Function	Purpose
`pthread_create()`	Create a new thread
`pthread_exit()`	Terminate the calling thread
`pthread_join()`	Wait for a thread to finish
`sigaction()`	Install a signal handler
`kill()`	Send a signal to a process or process group
`pause()`	Sleep until a signal is received
`pthread_kill()`	Send a signal to a specific thread
`pthread_sigmask()`	Change the calling thread's signal mask

Key idea

Threads share the process, but signal masks are per-thread. This makes signal handling in multithreaded programs powerful, but subtle.

Bibliography

Bibliography references for Linux process-related calls:

1. Linux man-pages project
   kill(2) — Linux manual page
   Sends a signal to a process or process group.
   https://man7.org/linux/man-pages/man2/kill.2.html
   pause(2) — Linux manual page
   Suspends the calling process until a signal is delivered.
   https://man7.org/linux/man-pages/man2/pause.2.html
   pthread_kill(3) — Linux manual page
   Sends a signal to a specific POSIX thread.
   https://man7.org/linux/man-pages/man3/pthread_kill.3.html
   pthread_create(3) — Linux manual page
   Creates a new POSIX thread.
   https://man7.org/linux/man-pages/man3/pthread_create.3.html
   pthread_exit(3) — Linux manual page
   Terminates the calling thread.
   https://man7.org/linux/man-pages/man3/pthread_exit.3.html
   pthread_join(3) — Linux manual page
   Waits for a thread to terminate.
   https://man7.org/linux/man-pages/man3/pthread_join.3.html
   pthread_sigmask(3) — Linux manual page
   Examines or changes the signal mask of the calling thread.
   https://man7.org/linux/man-pages/man3/pthread_sigmask.3.html
   signal-safety(7) — Linux manual page
   Lists functions that are async-signal-safe and discusses safe signal-handler programming.
   https://man7.org/linux/man-pages/man7/signal-safety.7.html
   signal(7) — Linux manual page
   Overview of signals, signal dispositions, masks, and signal handling in Linux.
   https://man7.org/linux/man-pages/man7/signal.7.html
   pthreads(7) — Linux manual page
   Overview of POSIX threads on Linux.
   https://man7.org/linux/man-pages/man7/pthreads.7.html
2. Neil Matthew, Richard Stones, Beginning Linux Programming, Wrox, 4th ed., 2007, ISBN-13 978-0470147627